

Structured feedback loops that actually fix agent mistakes

The single most important finding across all research is that diagnosis, not repair, is the bottleneck.

RefineBench (2025) showed frontier LLMs achieve near-perfect correction when given explicit, structured feedback about specific unmet criteria — but struggle to self-diagnose errors. (Emergent Mind) This means your instinct to move from freeform to structured feedback is well-supported: the more precisely your critic agent localizes and categorizes problems, the fewer correction iterations you'll need. Production systems and academic research converge on a prescriptive, multi-field feedback format with **error localization**, **category classification**, and **actionable fix instructions** as the three highest-impact fields. Severity, while intuitive, has not been directly studied in ablation — but it enables smart routing and prioritization. The optimal architecture for a LangGraph-based system combines structured feedback in graph state (checkpointed for crash recovery) with a persistent workspace file as ground truth and a summary injected as a user message on resume.

How production systems structure rejection feedback today

Production agent systems span a wide spectrum from binary signals to rich structured schemas, but a clear pattern emerges: **the more structured the feedback, the fewer iterations agents need.**

RLHF/RLAIF systems use surprisingly constrained formats. OpenAI's InstructGPT collects pairwise rankings of 4–9 outputs along dimensions like helpfulness, truthfulness, and harmlessness (GitHub) — not freeform essays. Anthropic's Constitutional AI goes further, decomposing "good behavior" into **16 named principles** randomly sampled per critique-revision cycle, each acting as a structured evaluation criterion. (NVIDIA) (Anthropic) The open-source ecosystem (TRL, Argilla, Label Studio) standardized on the {prompt, chosen, rejected} triplet for DPO training, (Hugging Face) (Second Talent) with Argilla adding typed question schemas — (RatingQuestion), (LabelQuestion), (RankingQuestion), and (SpanQuestion) — enabling multi-dimensional, field-level annotation. (Argilla) The consistent lesson: even training-time feedback benefits from decomposition into named dimensions rather than holistic judgments.

CI/CD coding agents rely on deterministic signals as implicit structured feedback. SWE-agent's Agent-Computer Interface automatically runs a linter after every edit; (Swe-agent) removing this single guardrail dropped performance from **18% to 10.3%** on SWE-bench. (NeurIPS) Aider pipes linter output and test stderr/stdout directly to the LLM with auto-retry (Aider) (though it can enter infinite token-burning loops without bounds). (GitHub) Devin follows a test-debug-fix cycle in a sandboxed VM, consuming CI check results as text — but **72% of its passing solutions took over 10 minutes**, suggesting multiple correction iterations. (Cognition) OpenHands uses an event stream architecture where every action produces a typed observation that feeds back into context. (arXiv) Across all these systems, the feedback format is essentially {exit_code, stdout, stderr, file_context} — structured by the execution environment, not by freeform human description.

Multi-agent frameworks vary significantly. AutoGen's critic pattern uses multi-dimensional scoring with explicit rubrics (Medium) — for example, scoring (bugs), (transformation), (encoding), and (aesthetics) each on a 1–10 scale with mandatory rationale per dimension and a concrete action list. (GitHub) CrewAI uses webhook-based human-in-the-loop with a {execution_id, task_id, human_feedback} schema, (CrewAI) supporting a Writer → Reviewer loop where scores below a threshold trigger retry with accumulated context. LangGraph's canonical pattern uses (interrupt()) to pause execution and (Command(resume=...)) to inject structured feedback as

JSON (LangChain) — supporting actions like "continue", "update", and "reject" with associated data payloads.

Medium

Structured feedback dramatically outperforms freeform — but no clean head-to-head exists

No paper has run a controlled experiment comparing a YAML schema with severity/location/expected fields against freeform text. However, the indirect evidence is overwhelming and consistent across multiple independent research groups.

The MAF paper (Nathani et al., EMNLP 2023) provides the closest analog. It decomposes feedback into specialized per-error-type modules (hallucination, mathematical errors, incorrect reasoning) rather than using a single generic critic. (arXiv) The result: up to **20% improvement in mathematical reasoning** and **18% in logical entailment** over single-module feedback. (ADS) Two iterations with decomposed MAF feedback matched or exceeded Self-Refine after four iterations with generic feedback — a **2x efficiency gain** from structure alone.

Self-Debugging (Chen et al., ICLR 2024) ran a clean ablation across three feedback types: simple ("the code is wrong, fix it"), unit test feedback (execution results + error messages + expected vs. actual), and code explanation (rubber duck debugging). Unit test feedback — which is effectively structured with implicit fields for location, error type, and expected behavior — achieved **+12.2%** on MBPP versus simple feedback's **+2.7%**. It also matched baselines generating **10x more candidates**, making it dramatically more cost-efficient.

Self-Refine (Madaan et al., NeurIPS 2023) confirmed through ablation that feedback quality is the primary driver of refinement quality. Its effective feedback format has two components: **(1) localization** of where the problem exists and **(2) actionable instruction** for what to change. (ResearchGate +2) Replacing task-specific feedback with generic "please improve" feedback degraded performance substantially. Random feedback performed worst of all.

The fields that matter most, ranked by evidence strength:

Field	Impact	Key evidence
Error localization (where)	Critical	Self-Refine, MAF, Self-Debug all require it
Error category (what type)	Very high	MAF's per-type modules deliver 2x efficiency
Expected behavior (what should happen)	High	Self-Debug's UT feedback includes expected vs. actual
Fix instruction (how to correct)	High	Self-Refine's two-component format; Hattie & Timperley's "feedforward"
External verification result (evidence)	Very high	Huang et al. show external signals are the only reliable path
Prior attempt history	High	Reflexion's episodic memory; Self-Refine retains full history
Severity	Not directly studied	Enables routing but no ablation exists

The optimal correction pattern: 2–3 bounded iterations with strategic fresh starts

Research converges on a clear diminishing-returns curve for iterative correction. The **Debugging Decay Index** (Adnan & Kuhn, Nature Scientific Reports 2025) models debugging effectiveness as exponential decay: $E(t) = E_0 \times e^{(-\lambda t)}$. Most models lose **60–80% of debugging capability within 2–3 attempts**. GPT-4-class models exhaust their debugging capacity by attempt 3, with a half-life of roughly 1–2 attempts. The paper's key recommendation: **strategic fresh starts** (restarting from scratch at the decay threshold) outperform continued iteration. [\(nature\)](#)

This aligns with findings across the field. Self-Refine shows biggest gains in iterations 1–2 with diminishing returns after 3–4. [\(Emergent Mind\)](#) Self-Debugging succeeds within **3 turns** despite allowing 10. Reflexion captures most gains in the first 3–5 trials out of 12. [\(OpenReview\)](#) The CRITIC framework typically converges in 1–3 rounds with external tool verification. [\(arXiv\)](#)

Single-pass correction (all feedback at once) works well when feedback is structured and comprehensive — the agent can address multiple issues in one generation. **Iterative refinement** is necessary when initial feedback is incomplete or when fixes introduce regressions. **Targeted correction** (re-processing only flagged sections) is most efficient when localization is precise, but the DDI paper warns that when targeted correction stalls, full regeneration is more effective than forcing more iterations.

ProActive Self-Refinement (PASR, 2025) introduces an interesting alternative: in-process, token-level decisions about what to refine rather than post-hoc correction. This yields **–42% token usage** and **+8.2 percentage points accuracy** over post-hoc methods [\(Emergent Mind\)](#) — suggesting that embedding correction awareness into the generation process itself may be the most cost-effective approach.

For practical implementation, the recommended pattern is:

- **Round 1:** Deliver all structured feedback at once; agent attempts comprehensive fix
 - **Round 2:** Targeted re-review of flagged issues only; catch remaining problems and regressions
 - **Round 3 (if needed):** Fresh start with accumulated learnings rather than further iteration on the same attempt
 - **Hard cap at 3 rounds** with automatic escalation to human review if unresolved
-

Prescriptive feedback with exemplars maximizes critic actionability

Educational psychology provides the strongest theoretical framework here. Hattie and Timperley's (2007) meta-analysis, validated by Wisniewski et al.'s (2020) follow-up across **435 studies and 61,000+ participants**, [\(PubMed Central\)](#) [\(Frontiers\)](#) identifies four levels of feedback effectiveness. The most effective combination answers three questions: **Where am I going?** (the goal/spec), **How am I doing?** (current gap), and **Where to next?** (concrete action). [\(ResearchGate\)](#) This maps directly to prescriptive feedback — what's wrong plus how to fix it.

Pure problem-focused feedback ("Line 42 has a null pointer risk") is task-level only — the least effective category above self-level praise. [\(PubMed Central\)](#) Balanced feedback (mixing positives with negatives) adds psychological safety but wastes tokens when communicating with agents rather than humans. **Prescriptive feedback** ("Line 42: null risk. Add `if (x != null)` guard before access") maps to process-level plus feedforward, the most effective pairing. **Exemplar-based feedback** ("Line 42: null risk. Correct pattern: `const val = x?.property ?? default`") is even stronger when feasible, providing a concrete template.

Constitutional AI validates this in practice: generating critiques **before** revisions improves harmlessness compared to generating revisions directly. [\(arXiv +2\)](#) The SiriuS framework (2025) adds an important architectural insight: **separate judgment from critique**. A Judgment Agent decides *whether* work needs correction (gatekeeper), while the Critic Agent provides feedback *only when correction is needed*. This prevents unnecessary modification of already-correct work [\(arXiv\)](#) — a real risk when critic agents are incentivized to find problems.

The recommended critic output schema, synthesized from CodeRabbit's production format, [\(GitHub\)](#) AutoGen's dimensional scoring, and the academic literature:

```
yaml
```

review:

verdict: "NEEDS_REVISION" # APPROVED | NEEDS_REVISION | REJECTED

confidence: 0.87

iteration: 1

max_iterations: 3

issues:

- id: "ISS-001"

severity: critical # critical | major | minor

category: correctness # correctness | performance | style | security | completeness

phase: "code_implementation"

location:

file: "src/handler.ts"

section: "data_processing"

lines: [42, 42]

problem: "Null pointer risk: user.address.city accessed without null check"

expected: "Graceful handling when address is null per UserSchema"

fix: "Add optional chaining: user?.address?.city ?? 'Unknown'"

exemplar: "See auth_handler.ts:87 for the correct pattern"

- id: "ISS-002"

severity: major

category: performance

phase: "code_implementation"

location:

file: "src/query.ts"

section: "query_builder"

lines: [15, 28]

problem: "N+1 query pattern in loop"

expected: "Single batched query"

fix: "Use WHERE id IN (...) instead of per-item queries"

exemplar: **null**

positive_observations:

- section: "authentication"

note: "JWT validation follows security best practices"

verification_checklist:

- check: "ISS-001 null guard added"

type: targeted

automated: **true**

- check: "ISS-002 batch query implemented"

type: targeted

automated: **true**

- check: "Auth tests still pass"

type: regression

automated: true

The `positive_observations` field is optional but useful — SiriuS showed that modifying all responses (including correct ones) degrades overall quality, so explicitly marking what's correct prevents the agent from "fixing" things that work. `arXiv`

Where feedback goes in context determines whether agents actually use it

The "Lost in the Middle" phenomenon (Liu et al., 2023, TACL 2024) is the single most important finding for feedback injection design. LLMs exhibit a **U-shaped attention curve**: highest accuracy when relevant information appears at the **beginning or end** of context, with **>30% performance degradation** when critical information is buried in the middle. This holds even for explicitly long-context models. `arXiv` `Maxim Articles`

For long-running multi-phase corrections, no single injection point is sufficient. The recommended layered approach for LangGraph:

Layer 1 — System prompt (cross-cutting rules only): Add 2–3 lines maximum for persistent behavioral corrections like "always validate inputs before processing" or "use TypeScript strict mode." System prompts are processed at the beginning of context with elevated priority. `Prompt Hub` But Claude places more weight on user messages than system prompts, `PromptLayer` and instruction bloat degrades compliance — Claude Code's system prompt already contains ~50 instructions, consuming nearly a third of instruction-following capacity.

Layer 2 — Graph state (current feedback items): Store the active feedback list, current phase, and iteration count in LangGraph's `TypedDict` state. This is automatically checkpointed, survives crashes, and the agent accesses it directly each step. `DataCamp +3` The canonical LangGraph pattern stores feedback as a state field that the generator checks with `if state.get("feedback")`. `LearnOpenCV` `Langchain` Keep state lean — avoid storing large artifacts, as LangGraph creates a checkpoint at every super-step. `Fast`

Layer 3 — Workspace file (full structured history): Write feedback to a `feedback.yaml` file in the agent's workspace. This is the **ground truth** that survives context window compaction (Claude Code's auto-compact triggers at 95% capacity and summarizes history, `Claude` but files persist), crash recovery, and long execution sequences. The agent can re-read this file at any point, avoiding the "lost in the middle" problem entirely. This pattern mirrors the proven `CLAUDE.md` / `.cursorrules` approach now standard in production code agents. `CodeRabbit` `coderrabbit`

Layer 4 — User message on resume (summary): When the agent resumes after rejection, inject a concise summary of the highest-priority feedback items as a user message. This exploits the recency bias — the most recent message sits at the end of context, receiving peak attention. But it fades as the correction sequence lengthens, which is why it must be backed by the workspace file.

For crash recovery specifically, LangGraph's checkpointing handles the mechanical state restoration `langchain` — interrupted threads can be resumed "many months later, on a different machine." `LangChain` The workspace file provides the semantic recovery: if context was compacted during a long correction, the file still contains the

full feedback history. Use completion markers (`.done`) files or phase flags in state) and idempotent operations to handle mid-correction crashes safely.

Quality assurance: a hybrid gate that reduces human burden by 70%+

After the agent applies corrections, the verification strategy should be a progressive gate that catches problems early and cheaply, escalating to expensive human review only when necessary.

Stage 1 — Automated pre-checks (instant to seconds): Run linting, type checking, schema validation, and formatting checks. SWE-agent's ablation showed that automatic linting alone accounts for a **7.7 percentage point improvement** in resolve rate. (NeurIPS) (Emergent Mind) These checks catch mechanical errors before wasting LLM compute or human attention. (Razorops)

Stage 2 — Targeted verification of flagged issues (seconds): For each issue in the feedback, run the specific check defined in the (verification_checklist). This catches whether the agent actually addressed each flagged problem. The SiriuS framework showed that targeted verification (checking only flagged items) preserves correct work better than re-evaluating everything, because full re-evaluation risks incorrectly flagging previously-approved sections. (arXiv)

Stage 3 — Regression test subset (seconds to minutes): Run tests covering areas adjacent to the changes, ensuring fixes didn't break previously correct functionality. Group tests by function and run only relevant suites per change. (xMatters)

Stage 4 — Confidence-based routing to human review: Implement progressive trust scoring. Cleanlab's TLM approach reduced incorrect responses by **10–56%** across five agent architectures by assigning real-time trust scores. (Cleanlab) The routing logic:

- Confidence $\geq 95\%$ → auto-approve
- 85–95% → light human spot-check
- 70–85% → full human review
- Below 70% → reject and regenerate with feedback

This progressive gate means humans only review the **residual uncertainty** after automated checks have cleared mechanical correctness. CodeRabbit's production pipeline adds a verification agent that validates each review comment before posting, filtering low-value feedback (CodeRabbit) — an approach worth replicating for critic agent outputs to reduce noise.

Bound the full QA loop to **2–3 cycles maximum**. Both the Debugging Decay Index research and multi-agent debate studies (Ueda et al., 2025) confirm saturation after 2–3 rounds of critique-revision. (Emergent Mind) (Swarnendu De) After 3 failed cycles, escalate to human intervention rather than continuing to burn compute.

Synthesis: the recommended architecture for LangGraph multi-phase jobs

For a LangGraph-based system with crash recovery, multi-phase jobs (research → analysis → code), and human/critic review gates, the optimal architecture combines the strongest patterns from production and research:

Feedback format: Use the structured YAML schema above with `severity`, `category`, `phase`, `location`, `problem`, `expected`, `fix`, and optional `exemplar` fields. Include a `verification_checklist` for each issue. This format directly addresses the three highest-impact feedback components identified in the literature (localization, categorization, actionable instruction) while adding the practical fields needed for automated verification and routing.

Correction flow: Implement a Generator → Critic → conditional routing loop in LangGraph (`Activewizards`) with a `max_iterations` counter in state. (`Activewizards`) Use `interrupt()` for human review gates at phase boundaries. (`LangChain +2`) Store feedback in state (checkpointed) and write to `feedback.yaml` in the workspace (persistent). On resume, inject a priority-sorted summary as the user message. Route corrections through the progressive QA gate before returning to the critic.

Key architectural decisions:

- **Separate judgment from critique** (SiriusS pattern): A lightweight check determines *whether* correction is needed before invoking the full critic, preventing unnecessary modification of correct work (`Toloka`) (`arXiv`)
- **Phase-tagged feedback:** Each feedback item references its target phase, enabling the agent to scope corrections appropriately and skip unaffected phases
- **Strategic fresh starts after round 3:** If the agent hasn't resolved issues after 3 iterations, regenerate from scratch with accumulated learnings rather than iterating further
- **Idempotent phase operations:** Use completion markers in state to handle mid-correction crashes, ensuring resumed agents don't re-execute completed phases (`Claude Code Plugins`)
- **Feedback compression:** Move resolved feedback items to a `resolved` section with one-line summaries; keep only active items in the primary feedback list to manage context window budget

This architecture turns the vague "please fix the issues" pattern into a machine-readable correction protocol where every feedback item has a clear location, category, fix instruction, and verification check — reducing average correction iterations from the 4+ typical of freeform feedback to the 1–2 that structured feedback enables.